

## Лекция 2. Сортировки: слияние и быстрая сортировка

---

## ГЛАВА 1.

# СОРТИРОВКИ: СЛИЯНИЕ И БЫСТРАЯ СОРТИРОВКА

---

### 1.1 Сортировка слиянием

**Определение 1.** (*merge sort*) Рекурсивный алгоритм «разделяй и властвуй»: массив делится пополам, каждая половина сортируется рекурсивно, затем две отсортированные половины сливаются (**merge**) в один отсортированный массив за линейное время.

Слияние двух отсортированных массивов длины  $n/2$  каждый делается за  $O(n)$  — идём по обоим одновременно и на каждом шаге забираем меньший из текущих элементов. Рекуррентное соотношение для времени работы:

$$T(n) = 2T(n/2) + O(n), \quad T(1) = O(1).$$

*Пример 1* — разбор рекурренты

По основной теореме о рекуррентных соотношениях (master theorem) при  $a = 2$ ,  $b = 2$ ,  $f(n) = O(n)$  получаем  $T(n) = O(n \log n)$  — и в худшем, и в среднем случае.

Сортировка слиянием **стабильна** (не меняет порядок равных элементов) и гарантирует  $O(n \log n)$  даже в худшем случае — в отличие от быстрой сортировки. Плата за это —  $O(n)$  дополнительной памяти на слияние.

### 1.2 Быстрая сортировка

**Определение 2.** (*quicksort*) Выбирается опорный элемент (**pivot**), массив разбивается на элементы меньше и больше опорного (партиционирование за  $O(n)$ ), затем обе части сортируются рекурсивно.

*Пример 2* — худший случай

Если опорным всегда выбирать первый элемент уже отсортированного массива, каждое разбиение даёт части размера 0 и  $n - 1$  — рекурсия вырождается в  $T(n) = T(n - 1) + O(n)$ , что даёт  $O(n^2)$ .

*Пример 3* — средний случай

При случайном выборе опорного элемента (или случайной перестановке входа) ожида-

емое время работы —  $O(n \log n)$ : асимптотически такое же, как у сортировки слиянием, но с меньшей константой и без дополнительной памяти.

### 1.3 Сравнение

| Алгоритм            | Худший случай | Средний случай | Память      |
|---------------------|---------------|----------------|-------------|
| Сортировка слиянием | $O(n \log n)$ | $O(n \log n)$  | $O(n)$      |
| Быстрая сортировка  | $O(n^2)$      | $O(n \log n)$  | $O(\log n)$ |

. . .

На практике быструю сортировку почти всегда предпочитают сортировке слиянием благодаря работе на месте (in-place) и меньшим константам — если только не нужна гарантия  $O(n \log n)$  в худшем случае или стабильность.